

Puzzlefin — System Design Document

Bil - destein@puzzle.io - Version 0.1, 2026-07-14

1

Table of Contents

1. Overview
2. Goals and Non-Goals
3. Architecture
 - 3.1. System Context
 - 3.2. Component View
4. Data Model
5. Deployment
6. Cross-Cutting Concerns
7. Open Questions
8. References

This document describes the architecture of the Puzzlefin system. It is a starter template that demonstrates AsciiDoc features — heading levels, an automatic table of contents, tables, admonitions, code blocks, images, and diagrams. Replace the placeholder text as you go.

1. Overview

Puzzlefin is a three-tier web application. This section explains what the system does and who uses it in plain prose.

You can format text with **bold**, *italic*, `monospace`, and add footnotes like this.^[1]

TIP

Admonition blocks (TIP, NOTE, IMPORTANT, WARNING, CAUTION) are a built-in AsciiDoc feature and render as nicely styled callouts.

2. Goals and Non-Goals

Goals

- Serve partner-facing API traffic reliably
- Provide a clean presentation tier for end users
- Persist data durably

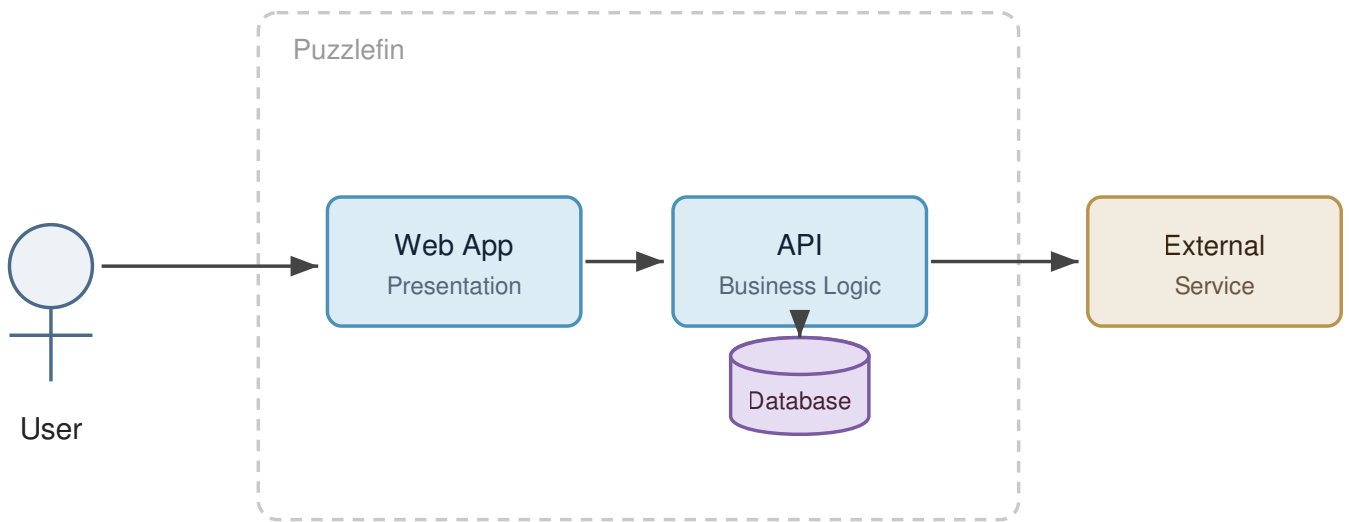
Non-Goals

- Real-time streaming analytics (out of scope for v1)
- Native mobile applications

3. Architecture

3.1. System Context

A short paragraph describing how the system sits relative to users and external services. The diagram below is defined **as code** and rendered by AsciiDoctor Diagram (see note).



NOTE

Diagram-as-code blocks (PlantUML/Mermaid) require the **Asciidoctor Diagram** extension or **Kroki**. In the VS Code AsciiDoc extension, enable `asciidoc.extensions.enableKroki` in Settings to render them in preview. If it doesn't render yet, see the image-based approach in Deployment.

3.2. Component View

Component	Responsibility	Technology
Web App	Presentation, client-side state	React
API	Business logic, auth, orchestration	Node / GraphQL
Database	Persistence	PostgreSQL

4. Data Model

Describe entities and relationships here. You can embed source code with syntax highlighting:

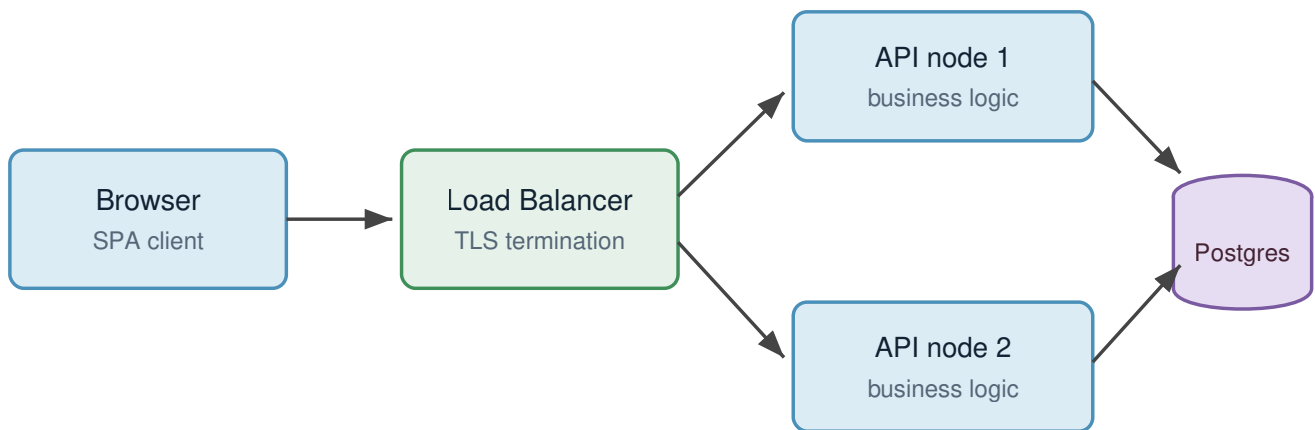
```
CREATE TABLE document (  
  id          UUID PRIMARY KEY,  
  title       TEXT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT now()  
);
```

SQL

5. Deployment

For freeform, Lucidchart-style pictures, draw them in draw.io, export an SVG into the `images/` folder, and reference it like this:

Deployment topology (illustrative)



WARNING

The image above will show a "not found" placeholder until you add `images/deployment-topology.svg`. That's expected in this starter.

6. Cross-Cutting Concerns

Security

OAuth 2.0 + OpenID Connect, FAPI-aligned for partner APIs.

Observability

Logging, metrics, tracing.

Reliability

SLOs, backups, failure modes.

7. Open Questions

- ❑ Question one
- ❑ Question two

8. References

- Link to related docs, RFCs, and tickets.

1. Footnotes render at the bottom of the document.